SELF-HEALING INFRASTRUCTURE

Automated Service Monitoring, Alerting and Recovery using Prometheus, Alertmanager and Ansible

Project	Self-Healing Infrastructure	**Platform**	AWS EC2 / Ubuntu
Monitoring	Prometheus + Blackbox Exporter	**Automation**	Alertmanager + Flask + Ansible

1. Introduction

Modern applications are expected to remain available and respond reliably. A service interruption can affect users and may require an administrator to identify the fault and restore the service manually. Self-healing infrastructure reduces this manual effort by combining continuous monitoring, alerting and automated recovery. When a monitored service becomes unavailable, the monitoring system detects the failure, an alert is raised, and an automation workflow attempts to restore the service.

2. Abstract

This project demonstrates a basic self-healing workflow for an NGINX web service hosted on an AWS EC2 Ubuntu instance. Docker is used to run the application and monitoring components. Prometheus collects probe results from Blackbox Exporter to check whether NGINX responds over HTTP. A Prometheus alert rule identifies a failed probe, and Alertmanager forwards the firing alert to a Python Flask webhook receiver. The receiver invokes a restricted Ansible playbook that restarts the NGINX container and verifies that it is running. The design illustrates how monitoring and configuration-management tools can work together to automate a simple recovery action.

3. Objectives

- Deploy an NGINX service in a Docker container on an AWS EC2 Ubuntu instance.
- Monitor HTTP availability continuously using Prometheus and Blackbox Exporter.
- Create an alert rule to detect when the NGINX health probe fails.
- Use Alertmanager and a Flask webhook to trigger an Ansible recovery playbook.
- Verify service availability after the recovery action.

4. Tools and Technologies Used

Tool	Purpose
AWS EC2 + Ubuntu	Cloud virtual machine hosting the project.
Docker	Runs NGINX and monitoring services in containers.
Prometheus	Collects probe metrics and evaluates alert rules.
Blackbox Exporter	Checks HTTP availability of the NGINX endpoint.
Alertmanager	Routes firing alerts to the webhook receiver.
Python Flask	Receives webhook notifications and starts recovery.
Ansible	Runs the playbook to restart and verify NGINX.

5. Implementation Procedure

1. Environment setup: Created an AWS EC2 Ubuntu instance, connected through SSH and installed the required tools. A Docker network named **monitoring** was used for communication between monitoring containers.

2. Application deployment: Deployed the NGINX application as a Docker container named **nginx-app**, exposing HTTP on port 80. The container was connected to the monitoring network.

3. Monitoring configuration: Deployed Prometheus and Blackbox Exporter. Prometheus was configured to send HTTP probe requests to **http://nginx-app:80** through Blackbox Exporter. The metric **probe_success** indicates probe status: 1 for success and 0 for failure.

4. Alert rule creation: Added a Prometheus alert rule named **NginxDown**. The expression **probe_success{job="nginx"} == 0** evaluates whether the NGINX probe has failed. A 15-second pending period helps avoid reacting to a very brief interruption.

5. Alert routing: Configured Alertmanager to receive alerts from Prometheus and send firing notifications to the Flask webhook endpoint. A bearer token is used to authenticate webhook requests.

6. Automated recovery: The Flask receiver validates the incoming token and alert status, then invokes a restricted Ansible playbook. The playbook restarts the NGINX Docker container and checks that its running state becomes true.

7. Validation: Validated the Prometheus configuration and alert rules, checked the Alertmanager readiness endpoint, and confirmed that the initial NGINX probe returned **probe_success = 1**. The failure simulation should be completed and its logs/screenshots attached as final evidence.

6. Working Principle

1. Detect Blackbox Exporter checks NGINX.	**2. Alert** Prometheus evaluates NginxDown.	**3. Notify** Alertmanager calls Flask.
4. Recover Flask invokes Ansible.	**5. Restart** Ansible restarts nginx-app.	**6. Verify** HTTP probe should return 1.

7. Results and Verification

The monitoring setup was verified: Prometheus reported the NGINX HTTP probe as **probe_success = 1**, the Prometheus alert-rule file passed validation, and the Alertmanager readiness endpoint returned OK. The Flask health endpoint also returned **Webhook receiver is running**. These checks confirm that the monitoring and webhook components were available during validation. The end-to-end failure-and-recovery test (stopping NGINX and confirming that the alert triggers Ansible and restores HTTP 200) must be recorded only after it has been observed successfully.

Verification status	Observed / required evidence
Monitoring and alert rules	Validated; initial NGINX probe returned 1.
Webhook health endpoint	Responded successfully.
End-to-end automatic recovery	Final failure simulation and recovery output still to be captured.

8. Conclusion

This project demonstrates the design of an automated monitoring and recovery workflow using open-source DevOps tools on AWS. Prometheus and Blackbox Exporter provide service health monitoring, Alertmanager routes failure notifications, and a Flask receiver connects the alert to an Ansible recovery playbook. The implementation provides practical experience with cloud virtual machines, containers, monitoring, alerting, webhooks and automation. Successful completion of the final failure simulation will demonstrate that the service can recover without a manual restart.

Note: Before submitting, add your name, college, internship organization and dates if required by your institution. Attach screenshots of the actual failure alert, webhook execution, Ansible result and recovered NGINX service after the final test.